Tutorial: Designing Classes

Rational and TokenScanner

CSC3002 C/C++ Programming

```
class Point {
public:
    Point();
    Point(int x0, int y0);
    int getX() const { return x; }
private:
    int x;
    int y;
};
```

Class: a type
`Point` describes data and operations.

Object: one instance
`p` is a particular point.

```
Point p(2, 3);
std::cout << p.getX();   // 2
```

Constructor definitions follow on slide 4.

Public and Private Members

`getX()` is public; `x` and `y` are private.

```
Point p(2, 3);
Point* pp = &p;
```

Expression in client code	Allowed?
`p.getX()`	Yes: call a public method.
`p.x`	No: `x` is private.
`pp->getX()`	Yes: access through a pointer.
`(*pp).getX()`	Yes: get the object, then call.
`pp.getX()`	No: `pp` is a pointer.

Default access in C++: `struct` is public; `class` is private.

A constructor gives an object its initial member values.

```
Point::Point() : x(0), y(0) {}

Point::Point(int x0, int y0)
    : x(x0), y(y0) {}
```

In the **member initializer list**, `x(0)` initializes member `x` to `0` **before the constructor body**.

```
Point p;          // Calls Point(): (0, 0)
Point q(2, 3);    // Calls Point(int, int): (2, 3)
Point r(2);       // Error: no matching constructor
```

Read the declarations and initializers

These constructors accept **zero or two arguments**.
`x(x0)` initializes member `x` from parameter `x0`.

Reading const Objects

`int getX() const` can be called on both objects below.

```
Point p(2, 3);
const Point q(4, 5);
std::cout << p.getX() << " " << q.getX();  // 2 4
```

Quick choice: A separate class `Tag` has a member `const int id;` with no in-class initializer. Which constructor definition works?

A

```
Tag::Tag(int n)
    : id(n) {}
```

B

```
Tag::Tag(int n) {
    id = n;
}
```

Explain your choice: **initialization** or **assignment**?

A fraction is **one value** represented by **two integers**.

$$\frac{1}{2} + \frac{1}{3} = \frac{5}{6}$$

```
Rational a(1, 2), b(1, 3);
Rational c = a + b;
std::cout << c.toString();  // 5/6
```

- Store numerator and denominator together.
- Put fraction rules in the constructor and methods.
- Reuse `a + b` wherever fraction addition is needed.

Our goal: use familiar arithmetic to learn **class design**.

Rational: Declarations and Definitions

`Rational.h`: declare the class

```
#include <string>

class Rational {
public:
    Rational(int n, int d);
    Rational operator+(
        const Rational& rhs) const;
    std::string toString() const;
private:
    int num;
    int den;
};
```

`Rational.h`
Class and member declarations.

`Rational.cpp`
Constructor and method definitions, using `Rational::`.

`main.cpp`
Create objects and call their public methods.

`public:` controls access. Bodies can be inside the class or defined outside.

`const Rational& rhs`: read `rhs`; trailing `const`: do not change this object.

`toString()`: returns integer text (`den == 1`) or fraction text (`num/den`).

Implementing the Rational Constructor

`Rational.cpp`: include `"Rational.h"` first. A constructor has **no return type**.

```
Rational::Rational(int n, int d)
    : num(n), den(d) {
    if (den == 0) {
        throw std::invalid_argument("zero denominator");
    }
    int g = gcd(std::abs(num), std::abs(den));
    num /= g;
    den /= g;
    if (den < 0) {
        num = -num;
        den = -den;
    }
}
```

`gcd` is a supplied helper. Check the denominator, reduce, then fix the sign.

Trace: $(6,-8) \to (3,-4) \to (-3,4)$ $\qquad$ $(0,5) \to (0,1)$.

Addition as a Member Function

`Rational.cpp`: define the public member declared in `Rational.h`.

Member call: `a + b` calls `a.operator+(b)`. $\frac{p}{q} + \frac{r}{s} = \frac{ps+rq}{qs}$

```
Rational Rational::operator+(const Rational& rhs) const {
    return Rational(num * rhs.den + rhs.num * den,
                    den * rhs.den);
}
```

`rhs` = **right-hand side**: the object `b`.
`const Rational&`: read `b` by reference.
Trailing `const`: cannot change `a`'s fields.
`num`, `den`: `a`; `rhs.num`, `rhs.den`: `b`.

First `Rational`: **return type**.
`Rational::` **class member**.

For $1/2 + 1/3$:
Numerator: $1 \times 3 + 1 \times 2 = 5$
Denominator: $2 \times 3 = 6$

A new result; both input values stay the same

Return `Rational(5, 6)`. Its constructor handles reduction.

FractionSum: Calculate and Update

`fraction_sum_main.cpp`: a complete client of `Rational`.

```
#include "Rational.h"
#include <iostream>

int main() {
    Rational sum(0, 1);
    sum = sum + Rational(1, 2);
    sum = sum + Rational(1, 3);
    sum = sum + Rational(1, 6);
    std::cout << sum.toString() << '\n';  // 1
}
```

Values of `sum`: $0 \longrightarrow \frac{1}{2} \longrightarrow \frac{5}{6} \longrightarrow 1$.
Two actions: `+` calculates a new fraction; `=` updates `sum`.

Build: `g++ -std=c++11 Rational.cpp fraction_sum_main.cpp -o sum`

Check: Does `sum + Rational(1, 2);` alone update `sum`?

Practice: Fractions and Access

```
Rational a(2, 4), b(1, 2);
Rational c = a + b;
std::cout << a.toString() << '\n';
std::cout << c.toString() << '\n';
```

1. What are the two printed lines?
2. Has `a` changed after the addition?
3. Would `a.den = 0;` compile in client code?

Take 90 seconds. Write the output and one short reason per answer.

Output

```
1/2
1
```

- Constructing `a` reduces $2/4$ to $1/2$.
- `c` stores $1/1$; `toString()` returns `"1"`.
- `a` stays $1/2$: addition returns a new object.

Access question

`a.den = 0;` in client code fails to compile: `den` is private.

A private-member access error and a zero-denominator input check answer different questions.

Tokens and the Scanning Loop

A **token** is one piece of text; it can contain several characters.

Here, `TokenScanner` comes from the **Stanford C++ Library** (`tokenscanner.h`).

```
TokenScanner scanner;
scanner.ignoreWhitespace();
scanner.setInput("ab + 12");

while (scanner.hasMoreTokens()) {
    std::string token = scanner.nextToken();
    std::cout << "[" << token << "] ";
}
```

```
[ab] [+] [12]
```

7 characters, 3 tokens: consecutive letters or digits stay together.

Why use it? Split expressions or commands into useful pieces.

The object remembers progress; scanning does **not evaluate** the expression.

A Scanner Class and Its Reading Position

Teaching model (not Stanford source). Full bodies are in the standalone program.

```
class SimplifiedTokenScanner {
public:
    SimplifiedTokenScanner()
        : pos(0), skipWhitespace(false) {}
    void setInput(const std::string& text);
    void ignoreWhitespace();
    bool hasMoreTokens();
    std::string nextToken();
private:
    std::string input;
    std::size_t pos;
    bool skipWhitespace;
    bool isSpace(unsigned char c);
    bool isWord(unsigned char c);
};
```

`input`: the saved input text.
`pos`: next **character index** to check.
`skipWhitespace`: skip whitespace.

Input `ab+`: length 3.

Call	Returns	pos after
`setInput("ab+")`	–	0
`hasMoreTokens()`	`true`	0
`nextToken()`	`"ab"`	2
`nextToken()`	`"+"`	3
`hasMoreTokens()`	`false`	3
`nextToken()`	`""`	3

Reading `ab`: `pos` advances by **2**.

`hasMoreTokens()` may skip ignored whitespace, but does not consume the next token.

Useful Scanner Methods

Stanford library: each row uses a fresh `TokenScanner s`; inputs show actual text.

Method	Input / setup	Example result
`hasMoreTokens()`	`x`; check twice.	`true`, `true`. Read `x`; next check is `false`.
`nextToken()`	`ab+`; read repeatedly.	`"ab"`, then `"+"`, then `""`.
`ignoreWhitespace()`	`a b`; compare new scanners.	Default: `"a"`, `" "`, `"b"`. With this option: `"a"`, `"b"`.
`scanNumbers()`	`12.5`; enable, then read.	One token: `"12.5"`.
`scanStrings()`	`"hi there"`; enable, then read.	One token: `"hi there"`. The quotes and internal space stay.
`saveToken(t)`	`x+y`; read `x` into `t`, then save `t`.	Next read returns `"x"` again, then reading continues with `"+"`.

`ab` **is one token.** `" "` contains one space; `""` means no token remains.

Practice: What Comes Next?

```
TokenScanner scanner;
scanner.ignoreWhitespace();
scanner.setInput("total = 7 + 2;");
std::string first = scanner.nextToken();

while (scanner.hasMoreTokens()) {
    std::cout << "[" << scanner.nextToken() << "] ";
}
```

1. What is stored in `first`?
2. Which tokens does the loop print, in order?
3. Does this code calculate `7 + 2` as 9?

Take 90 seconds. Track reading and printing separately.

Stored in `first`: `"total"`

Printed by the loop:

```
[=] [7] [+] [2] [;]
```

- `total` was read before the loop and was never printed.
- Reading tokens does not evaluate `7 + 2`.
- At the end: no tokens remain; another read returns `""`.

Three different actions

`hasMoreTokens()`: check `nextToken()`: read `cout`: print

Calling `hasMoreTokens()` again does not lose the next token.

Class Design Recap and Questions

1. A class groups **data and operations**.
2. **Public** methods are the client's interface; **private** members hold the internal representation.
3. **Declarations** describe operations; **definitions** provide their code.

Rational: design a class

Store a fraction.
Initialize and reduce in the constructor.
Return a new value from addition.

Scanner: remember progress

Store input and position.
Check and read tokens.
Keep the reading state private.

Questions?